

LVM (logical volume manager)

3 layers:

1. Physical Volume: raw disk
2. Volume Group: a pool of PVs, forming one big chunk of storage
3. Logical Volume: Virtual partition on VGs, what is actually formatted and mounted

Sudo pvs

Sudo vgs

Sudo lvs

For detailed info:

Sudo pvdisplay

Sudo vgdisplay

Sudo lvdisplay

Helps in resizing w/o downtime (no rebooting/remounting)

```
zarmeena@z:~$ sudo vgs
VG          #PV #LV #SN Attr   VSize   VFree
ubuntu-vg    1   1   0 wz--n- <23.00g 11.50g
```

Extend lv size

Sudo lvextend -L +2G /dev/ubuntu-vg/ubuntu-lv

```
zarmeena@z:~$ sudo lvextend -L +2G /dev/ubuntu-vg/ubuntu-lv
Size of logical volume ubuntu-vg/ubuntu-lv changed from <11.50 GiB (2943 exten
ts) to <13.50 GiB (3455 extents).
Logical volume ubuntu-vg/ubuntu-lv successfully resized.
```

NOW, you also need to extend the filesystem on top

Sudo resize2fs /dev/ubuntu-vg/ubuntu-lv

```
zarmeena@z:~$ sudo resize2fs /dev/ubuntu-vg/ubuntu-lv
resize2fs 1.47.0 (5-Feb-2023)
Filesystem at /dev/ubuntu-vg/ubuntu-lv is mounted on /; on-line resizing require
d
old_desc_blocks = 2, new_desc_blocks = 2
The filesystem on /dev/ubuntu-vg/ubuntu-lv is now 3537920 (4k) blocks long.
```

resize2fs : grows ext4

Ext4:

verifying

```
zarmeena@z:~$ df -h /
Filesystem              Size  Used Avail Use% Mounted on
/dev/mapper/ubuntu--vg-ubuntu--lv 14G   5.8G   7.3G  42% /
```

New logical volume

Sudo lvcreate -L +1G -n testlv ubuntu-vg (created lv)

Sudo mkfs.ext4 /dev/ubuntu-vg/testlv (created fs)

Sudo mkdir /mnt/testlv

Sudo mount /dev/ubuntu-vg/testlv /mnt/testlv (formatting + mounting)

Df -h /mnt/testlv

```
zarmeena@z:~$ sudo lvcreate -L 1G -n testlv ubuntu-vg
  Logical volume "testlv" created.
zarmeena@z:~$ sudo mkfs.ext4 /dev/ubuntu-vg/testlv
mke2fs 1.47.0 (5-Feb-2023)
Creating filesystem with 262144 4k blocks and 65536 inodes
Filesystem UUID: b94c03f5-a547-4572-8615-a25afba10327
Superblock backups stored on blocks:
    32768, 98304, 163840, 229376

Allocating group tables: done
Writing inode tables: done
Creating journal (8192 blocks): done
Writing superblocks and filesystem accounting information: done

zarmeena@z:~$ sudo mkdir /mnt/tstlv
zarmeena@z:~$ sudo mkdir /mnt/testlv
zarmeena@z:~$ sudo mount /dev/ubuntu-vg/testlv /mnt/testlv
zarmeena@z:~$ df -h /mnt/testlv
Filesystem                                Size  Used Avail Use% Mounted on
/dev/mapper/ubuntu--vg-testlv             974M   24K  907M   1% /mnt/testlv
```

Cleanup

Sudo umount /mnt/testlv

Sudo lvremove /dev/mnt/ubuntu-vg/testlv

```
zarmeena@z:~$ sudo umount /mnt/testlv
zarmeena@z:~$ sudo lvremove /dev/ubuntu-vg/testlv
Do you really want to remove and DISCARD active logical volume ubuntu-vg/testlv?
[y/n]: y
  Logical volume "testlv" successfully removed.
```

FIREWALL

Ufw is the simple frontend version, actual stuff is ip tables (low level kernel, configures firewall)

Sudo iptables -L -n -v

-L: list rules

-n: show numerics
-v: verbose

```
hain ufw-user-logging-forward (0 references)
pkts bytes target      prot opt in      out     source      destination

hain ufw-user-logging-input (0 references)
pkts bytes target      prot opt in      out     source      destination

hain ufw-user-logging-output (0 references)
pkts bytes target      prot opt in      out     source      destination

hain ufw-user-output (1 references)
pkts bytes target      prot opt in      out     source      destination
```

Basically ufw's rules translated into raw iptables

Iptables

Tables - organizes tables into rules based on purpose

Chains - within tables, represent direction of traffic: input/output/forward(passing through)

Rules - packet is checked against set of rules, top to bottom, first match wins

To write a raw iptable rule (not persistent, vanish when rebooted, unless saved)

Sudo iptables -A INPUT -p tcp --dport 8080 -j ACCEPT (or DROP or REJECT)

-A INPUT: appends to the INPUT chain, -t tcp protocol, --dport 8080 destination port, -j
ACCEPT the action (jumps to target)

DROP discards silently

REJECT sends back explicit rejection msg

Saving da rule

Sudo apt install iptables-persistent -y

Sudo netfilter-persistent save

To remove just use DROP

Sudo iptables -D INPUT -p tcp --dport 8080 -j DROP

OR

Check out rule no. and delete via rule no.

Sudo iptables -L INPUT -n -v --line-numbers

Sudo iptables -D INPUT 9(or wtv the no. is)

Verify

Sudo iptables -L INPUT -n -v

Rate limiting

Sudo ufw limit ssh

1. Tracks incoming interactions per ip address
2. Drops further attempts made by an ip if it has made xyz attempts in abc time
3. After cooling down period, ip allowed again

Check out the rules

Sudo iptables -L -n -v | grep -A 5 "22"

Make your own rules

```
sudo iptables -A INPUT -p tcp --dport 22 -m conntrack --ctstate NEW -m limit --limit 6/min --limit-burst 6 -j ACCEPT
```

-m limit: loads the rate limiting module

--limit 6/min: allowed rate

--limit-burst 6: immediate attempts allowed

If within limit, accepted

```
sudo iptables -A INPUT -p tcp --dport 22 -j DROP
```

Anything that escapes rule 1 gets dropped

Mysql version: 8.0.46

Wordpress framework + mysql integration

Installed wp

Connected mysql

Created user

```
zarmeena@z:/var/www/html$ curl -I http://localhost/
HTTP/1.1 302 Found
Server: nginx/1.24.0 (Ubuntu)
Date: Wed, 23 Sep 2026 02:03:33 GMT
Content-Type: text/html; charset=UTF-8
Connection: keep-alive
Expires: Wed, 11 Jan 1984 05:00:00 GMT
Cache-Control: no-cache, must-revalidate, max-age=0, no-store, private
X-Redirect-By: WordPress
Location: http://localhost/wp-admin/install.php
```

Multiple Stacks Side-by-Side ft Docker

Installed docker

Pulling pre-built images

```
docker run -d --name php81-stack -p 8081:80 php:8.1-apache
```

```
docker run -d --name php83-stack -p 8083:80 php:8.3-apache
```

-d: detached

--name: for humans

-p 8081:80 : maps host port 8081 to containers 80

```
zarmeena@z:~$ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS
05bcfc825533	php:8.3-apache	"docker-php-entrypoi..."	7 seconds ago	Up 6 se
cond5	0.0.0.0:8083->80/tcp, [::]:8083->80/tcp	php83-stack		
16efb179aa4c	php:8.1-apache	"docker-php-entrypoi..."	9 minutes ago	Up 9 mi
nutes	0.0.0.0:8081->80/tcp, [::]:8081->80/tcp	php81-stack		

```
curl http://localhost:8081
```

```
curl http://localhost:8083
```

```
zarmeena@z:~$ curl -v http://localhost:8081
* Host localhost:8081 was resolved.
* IPv6: ::1
* IPv4: 127.0.0.1
* Trying [::1]:8081...
* Connected to localhost (::1) port 8081
> GET / HTTP/1.1
> Host: localhost:8081
> User-Agent: curl/8.5.0
> Accept: */*
>
< HTTP/1.1 200 OK
< Date: Wed, 23 Sep 2026 05:05:48 GMT
< Server: Apache/2.4.65 (Debian)
< X-Powered-By: PHP/8.1.34
< Content-Length: 0
< Content-Type: text/html; charset=UTF-8
<
* Connection #0 to host localhost left intact
zarmeena@z:~$
```

```
zarmeena@z:~$ curl http://localhost:8083
PHP 8.3 container says: 8.3.33zarmeena@z:~$ curl -v http://localhost:8083
* Host localhost:8083 was resolved.
* IPv6: ::1
* IPv4: 127.0.0.1
* Trying [::1]:8083...
* Connected to localhost (::1) port 8083
> GET / HTTP/1.1
> Host: localhost:8083
> User-Agent: curl/8.5.0
> Accept: */*
>
< HTTP/1.1 200 OK
< Date: Wed, 23 Sep 2026 05:10:32 GMT
< Server: Apache/2.4.68 (Debian)
< X-Powered-By: PHP/8.3.33
< Content-Length: 30
< Content-Type: text/html; charset=UTF-8
<
* Connection #0 to host localhost left intact
PHP 8.3 container says: 8.3.33zarmeena@z:~$ _
```

NGINX NAMED BASED BASED ROUTING (virtual hosts)

Server-name + server blocks.

Nginx picks which block handles requests based on incoming host: http header against each block's server-name

```
server {
    listen 80;
    server_name site1.local;
    root /var/www/site1;
    index index.html;
}
```

```
sudo mkdir -p /var/www/site1 /var/www/site2
```

```
echo "<h1>This is Site 1</h1>" | sudo tee /var/www/site1/index.html
```

```
echo "<h1>This is Site 2</h1>" | sudo tee /var/www/site2/index.html
```

```
sudo nano /etc/nginx/sites-available/multisite
```

```
sudo ln -s /etc/nginx/sites-available/multisite /etc/nginx/sites-enabled/
```

```
sudo nginx -t
```

```
zarmeena@z:~$ sudo nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
```

```
sudo systemctl restart nginx
```

Fake host header -H for testing

```
curl -H "Host: site1.local" http://localhost
```

```
curl -H "Host: site2.local" http://localhost
```

JAVA SPRINGBOOT - DEVOPS DEPLOYMENT PERSPECTIVE

Installing java and maven

```
sudo apt update
```

```
sudo apt install openjdk-17-jdk maven -y
```

```
java -version (17.0.20.1)
```

```
mvn -version (3.8.7)
```

Using spring boot's official project generator to get project

```
cd ~
```

```
curl https://start.spring.io/starter.zip -d dependencies=web -d type=maven-project -d javaVersion=17 -d name=demoapp -o demoapp.zip
```

```
zarmeena@z:~$ curl https://start.spring.io/starter.zip -d dependencies=web -d type=maven-project -d javaVersion=17 -d name=demoapp -o demoapp.zip
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100 14917  100 14854  100    63   13697    58   0:00:01  0:00:01 --:--:-- 13761
```

```
sudo apt install unzip -y
```

```
unzip demoapp.zip -d demoapp
```

```
cd demoapp
```

Endpoint configure

```
nano src/main/java/com/example/demoapp/DemoappApplication.java
```

Adding controller, building and running

```
mvn clean package
```

(builds the jar package)

```
java -jar target/demoapp-0.0.1-SNAPSHOT.jar
```

To test

```
curl http://localhost:8080/
```

DB replication vs backup, and the BCP angle

Both solve different problems. Need them both

In replication, it makes copy of database in sync with work

Recovery Point Objective: how much data you lose. Here it's everything since the last backup. Backing up more often means a smaller RPO.

Recovery Time Objective: how long recovery takes. Here it's a few seconds, but on a big production database a restore can take hours.

Replication protects you when a server dies. Backups protect you when data gets damaged or deleted. Real businesses need both

```
mysql> CREATE DATABASE bcp;
Query OK, 1 row affected (0.07 sec)

mysql> USE bcp;
Database changed
mysql> CREATE TABLE orders (id INT AUTO_INCREMENT PRIMARY KEY, customer VARCHAR(50), city VARCHAR(30));
Query OK, 0 rows affected (0.11 sec)

mysql> INSERT INTO orders (customer, city) VALUES ('A','Lahore'), ('B','Bench'), ('C','meow'), ('D','delph
Query OK, 4 rows affected (0.05 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM orders;
+-----+-----+-----+
| id | customer | city |
+-----+-----+-----+
| 1 | A        | Lahore |
| 2 | B        | Bench |
| 3 | C        | meow |
| 4 | D        | delph |
+-----+-----+-----+
4 rows in set (0.00 sec)

mysql> EXIT;
Bye
zarmeena@z:~$ _
```

Backups

```
zarmeena@z:~$ mkdir -p ~/backups
zarmeena@z:~$ sudo mysqldump bcp > ~/backups/bcp.sql
zarmeena@z:~$ ls -lh ~/backup
ls: cannot access '/home/zarmeena/backup': No such file or directory
zarmeena@z:~$ ls -lh ~/backups/
total 4.0K
-rw-rw-r-- 1 zarmeena zarmeena 2.0K Sep 23 06:30 bcp.sql
zarmeena@z:~$
```

Creating breach

```
zarmeena@z:~$ sudo mysql bcp -e "DROP TABLE orders;"
zarmeena@z:~$ sudo mysql bcp -e "SELECT * FROM orders;"
ERROR 1146 (42S02) at line 1: Table 'bcp.orders' doesn't exist
zarmeena@z:~$
```

Solving

```

zarmeena@z:~$ sudo mysql bcp < ~/backups/bcp.sql
zarmeena@z:~$ sudo mysql bcp -e "SELECT * FROM orders;"
+-----+-----+-----+
| id | customer | city |
+-----+-----+-----+
| 1 | A | Lahore |
| 2 | B | Bench |
| 3 | C | meow |
| 4 | D | delph |
+-----+-----+-----+
zarmeena@z:~$ _

```

Can be optimized using cron jobs

Triggers, Routines & Indexes

Triggers(for audit trails, upon INSERT/UPDATE/DELETE)

```

mysql> USE bcp;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> CREATE TABLE audit ( customer VARCHAR(50), deletedby VARCHAR(100), deletedat DATETIME);
Query OK, 0 rows affected (0.10 sec)

mysql> CREATE TRIGGER logdlt AFTER DELETE ON orders FOR EACH ROW INSERT INTO audit VALUES (OLD.customer, U:
Query OK, 0 rows affected (0.05 sec)

mysql> DELETE FROM orders WHERE customer = 'A';
Query OK, 1 row affected (0.03 sec)

mysql> SELECT * FROM orders;
+-----+-----+-----+
| id | customer | city |
+-----+-----+-----+
| 2 | B | Bench |
| 3 | C | meow |
| 4 | D | delph |
+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT * FROM audit;
+-----+-----+-----+
| customer | deletedby | deletedat |
+-----+-----+-----+
| A | root@localhost | 2026-09-23 06:39:35 |
+-----+-----+-----+
1 row in set (0.00 sec)

mysql>

```

Routines (stored procedure)

```
mysql> CREATE PROCEDURE abc (IN c VARCHAR(30)) SELECT * FROM orders WHERE customer = c;
Query OK, 0 rows affected (0.05 sec)

mysql> CALL abc('c');
+----+-----+-----+
| id | customer | city |
+----+-----+-----+
| 3 | C        | meow |
+----+-----+-----+
1 row in set (0.02 sec)

Query OK, 0 rows affected (0.02 sec)

mysql>
```

Indexes (fast reading, slow writing)

```
mysql> EXPLAIN SELECT * FROM orders WHERE customer = 'c';
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE      | orders | NULL       | ALL  | NULL         | NULL | NULL    | NULL | 3    | 33.33    |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.01 sec)

mysql> CREATE INDEX idx_city ON orders(customer);
Query OK, 0 rows affected (0.11 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> EXPLAIN SELECT * FROM orders WHERE customer = 'c';
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE      | orders | NULL       | ref  | idx_city      | idx_city | 203    | const | 1    | 100      |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)

mysql> SHOW INDEX FROM orders;
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| Table | Non_unique | Key_name | Seq_in_index | Column_name | Collation | Cardinality | Sub_part | Packer |
| Comment | Index_comment | Visible | Expression |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| orders | 0 | PRIMARY | 1 | id | A | 3 | NULL | NULL |
| orders | 1 | idx_city | 1 | customer | A | 3 | NULL | NULL |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
2 rows in set (0.03 sec)
```

MySQL Replication Between 2 Instances (with Docker)

Creating a network to start both servers

```
docker network create repl-net
```

```
docker run -d --name mysql-primary --network repl-net -e
MYSQL_ROOT_PASSWORD=rootpass mysql:8.0 --server-id=1 --log-bin=mysql-bin
--gtid-mode=ON --enforce-gtid-consistency=ON --performance-schema=OFF
--innodb-buffer-pool-size=64M
```

```
docker run -d --name mysql-replica --network repl-net -e
MYSQL_ROOT_PASSWORD=rootpass mysql:8.0 --server-id=2 --gtid-mode=ON
--enforce-gtid-consistency=ON --read-only=ON --performance-schema=OFF
--innodb-buffer-pool-size=64M
```

--server-id: gives each server a unique ID
--log-bin: turns on the binary log on the primary.
--gtid-mode=ON: gives every transaction a unique ID
--read-only=ON: on the replica stops normal users from writing to it.

Checked logs

Creating a user on primary site

```
docker exec -it mysql-primary mysql -uroot -prootpass
```

```
CREATE USER 'repl'@'%' IDENTIFIED BY 'replpass';
GRANT REPLICATION SLAVE ON *.* TO 'repl'@'%';
EXIT;
(the account the replica uses to log in and pull changes.)
```

```
docker exec -it mysql-replica mysql -uroot -prootpass
```

```
CHANGE REPLICATION SOURCE TO
SOURCE_HOST='mysql-primary',
SOURCE_USER='repl',
SOURCE_PASSWORD='replpass',
SOURCE_AUTO_POSITION=1,
GET_SOURCE_PUBLIC_KEY=1;
```

```
START REPLICA;
SHOW REPLICA STATUS\G
```

Now Checking

```
Replica_IO_Running: Yes
Replica_SQL_Running: Yes
Seconds_Behind_Source: 0
```

Relationship not a replica

```
docker exec mysql-primary mysql -uroot -prootpass -e "DROP TABLE shop.orders;"
docker exec mysql-replica mysql -uroot -prootpass -e "SELECT * FROM shop.orders;"
```

Server Failure:

`docker stop mysql-primary`

`docker exec mysql-replica mysql -uroot -prootpass -e "SHOW REPLICA STATUS\G" | grep -E "Running|Error"`

Cleanup:

`docker stop mysql-replica`

`free -h`

```
mysql-replica
zarmeena@z:~$ docker stop mysql-primary
mysql-primary
zarmeena@z:~$ free -h
              total        used        free      shared  buff/cache   available
Mem:           2.8Gi         829Mi        241Mi        6.8Mi        2.0Gi        2.0Gi
Swap:          2.0Gi          524Ki        2.0Gi
zarmeena@z:~$
```

Storage engine

Incase of mysql, there are 2 layers

The top layer is for understanding sql

The bottom layer stores the data on disk, reads it back, and handles locking and crashes.

InnoDB shows DEFAULT, and its row says it supports transactions, row-level locking and foreign keys

ACID properties

Atomacity

Consistency

Isolation

Durability

Row-level locking

One update block everyone

Monitoring

In-box monitoring(tools run on machine to see its state)

Top, free -h, df, -h, iostat (shows disks read and write per sec) or simply vmstat

Lack of mointering, if server dies, this dies too, doesnt scale

Out-box monitering(agent runs on servers, collect values and sends to central monitoring system)

Push: the agent sends data to the central server by itself(elk)

Pull: the central server visits each agent and collects the data (prometheus)

Prometheus

Agents called exporters

```
zarmeena@z:~$ curl -s 'localhost:9090/api/v1/query?query=up'
{"status": "success", "data": {"resultType": "vector", "result": [{"metric": {"__name__": "up", "instance": "localhost:9090"}, "value": [1790148901.628, "1"]}, {"metric": {"__name__": "up", "instance": "localhost:9100", "job": "node"}, "value": [1790148901.628, "1"]}]}}zarmeena@z:~$
```

On windows browser

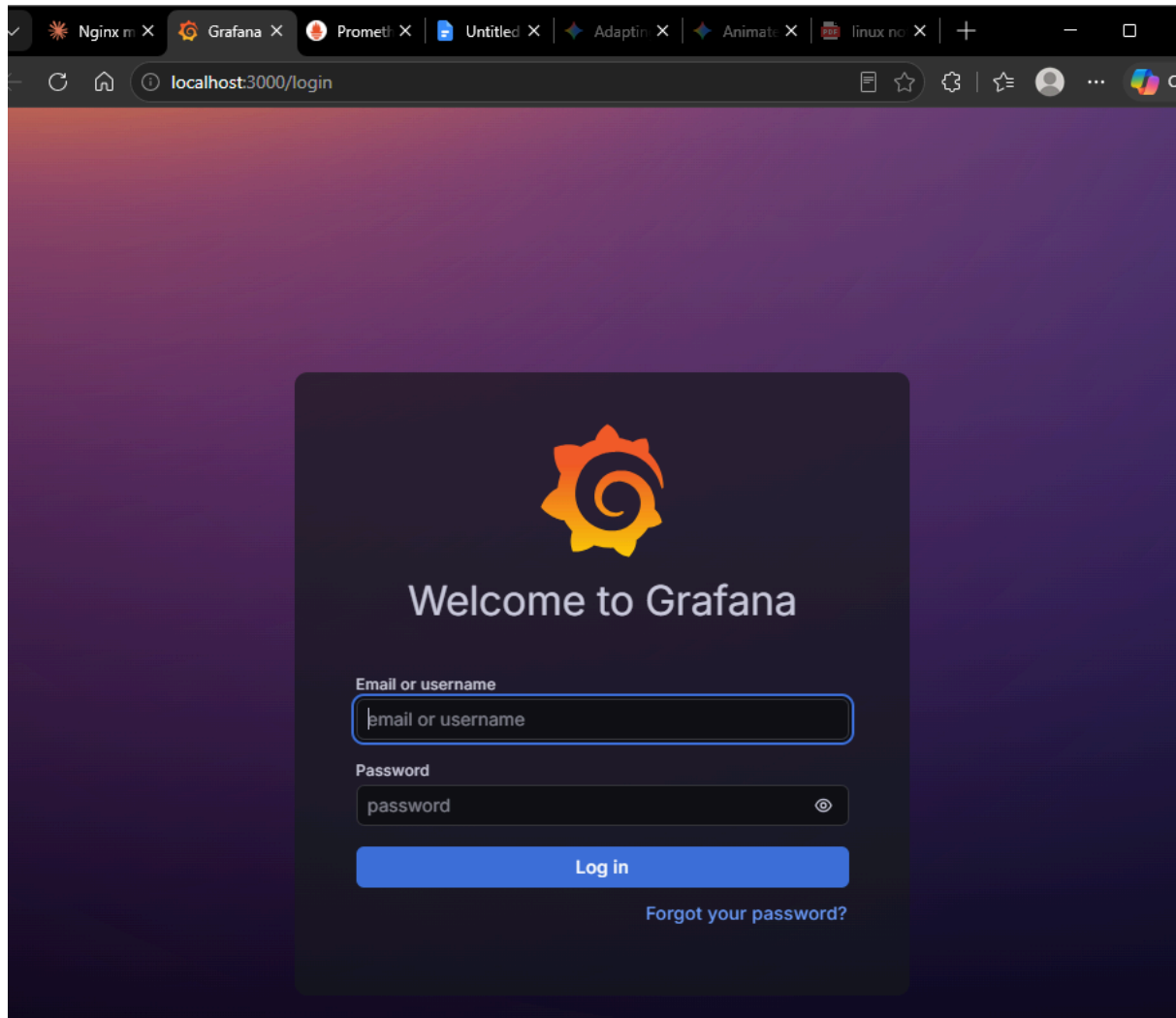
The screenshot shows the Prometheus web interface. At the top is a navigation bar with 'Prometheus', 'Alerts', 'Graph', 'Status', and 'Help'. Below this is a section for 'Enable query history' with a checkbox. A text input field contains the expression 'Expression (press Shift+Enter for newlines)'. To the left of the input is a blue 'Execute' button, and to the right is a dropdown menu showing '- insert metric at cursor -'. A 'Remove Graph' link is on the right. Below the input is a 'Graph' tab and a 'Console' tab. The 'Graph' tab is active, showing a 'Moment' time range selector with left and right arrow buttons. Below the selector is a table with two columns: 'Element' and 'Value'. The table is currently empty, showing 'no data'. At the bottom left is a blue 'Add Graph' button.

Grafana

Draw dashboards after taking in insights from prometheus etc

```
zarmeena@z:~$ curl -s localhost:3000/api/health
{
  "database": "ok",
  "version": "13.0.2",
  "commit": "3fcd5a"
}zarmeena@z:~$
```

On windows browser



ELK(elastic, logsearch, kibana)

Built for logs, agent called beats

Elastic search stores logs and make em searchable in secs

Logsearch parses and cleans logs

Kibana for dashboards